Cambios para garantizar el orden de consumo de eventos

Repositorio: `github.com/Juanpapb0401/cqrs-event-sourcing-kafka` – commit `6e9d8a7` sobre `jrquinte/cqrs-event-sourcing-kafka`.

1. Problema original

El producer publicaba cada evento en un topic distinto (`AccountOpenedEvent`, `FundsDepositedEvent`, `FundsWithdrawnEvent`, `AccountClosedEvent`) y el query service tenía un `@KafkaListener` por topic. Kafka solo garantiza orden dentro de una partición de un topic, nunca entre topics. Por eso un `FundsDeposited` podía procesarse antes que el `AccountOpened` de su cuenta; el handler no encontraba la cuenta, hacía `return` y el depósito se perdía en el read model.

2. Solución

Un solo topic, `BankAccountEvents`, y un solo método consumidor. Todos los eventos pasan por el mismo log ordenado.

Archivo	Cambio
account.cmd/application.yml	Nueva propiedad `spring.kafka.topic: BankAccountEvents`
account.query/application.yml	Misma propiedad para el consumer
AccountEventStore	Inyecta el topic con `@Value` y publica en él (antes usaba el nombre de clase del evento)
AccountEventSourcingHandler	Igual, incluyendo `republishEvents` (RestoreReadDb)
AccountEventProducer	Envía con key = id de la cuenta: todos los eventos de una cuenta van a la misma partición, aunque el topic tenga varias
EventConsumer	Una sola firma: `consume(BaseEvent event, Acknowledgment ack)`
AccountEventConsumer	Un `@KafkaListener(topics="${spring.kafka.topic}")` que despacha por reflexión a `EventHandler.on(<tipo concreto>)` y registra cada evento consumido. Si el handler falla no hace ack, así el evento se reintenta sin saltar el orden
AccountEventHandler	Corrección extra de un bug previo: `on(FundsWithdrawnEvent)` calculaba el balance pero no hacía `setBalance`
docker-compose.yml	Extra: Kafka, Zookeeper, MongoDB y MySQL (el repo no lo trae)

3. Por qué funciona

El `JsonSerializer` añade el header `__TypeId__` con la clase del evento y el `JsonDeserializer` lo usa para reconstruir la subclase concreta de `BaseEvent`. Así un único listener recibe todos los tipos, y como hay un único topic con la key = id de cuenta, Kafka entrega los eventos de cada cuenta en el mismo orden en que se escribieron.

4. Diff completo

bank-account/account.cmd/src/main/java/com/techbank/account/cmd/infrastructure/AccountEventProducer.java

```
@@ -13,6 +13,6 @@ public class AccountEventProducer implements EventProducer {

     @Override
     public void produce(String topic, BaseEvent event) {
-        this.kafkaTemplate.send(topic, event);
+        this.kafkaTemplate.send(topic, event.getId(), event);
     }
 }
```

bank-account/account.cmd/src/main/java/com/techbank/account/cmd/infrastructure/AccountEventSourcingHandler.java

```
@@ -6,12 +6,16 @@ import com.techbank.cqrs.core.handlers.EventSourcingHandler;
 import com.techbank.cqrs.core.infrastructure.EventStore;
 import com.techbank.cqrs.core.producers.EventProducer;
 import org.springframework.beans.factory.annotation.Autowired;
+import org.springframework.beans.factory.annotation.Value;
 import org.springframework.stereotype.Service;

 import java.util.Comparator;

 @Service
 public class AccountEventSourcingHandler implements EventSourcingHandler<AccountAggregate> {
+    @Value("${spring.kafka.topic}")
+    private String topic;
+
     @Autowired
     private EventStore eventStore;

@@ -44,7 +48,7 @@ public class AccountEventSourcingHandler implements EventSourcingHandler<Account
             if (aggregate == null || !aggregate.getActive()) continue;
             var events = eventStore.getEvents(aggregateId);
             for(var event: events) {
-                eventProducer.produce(event.getClass().getSimpleName(), event);
+                eventProducer.produce(topic, event);
```

```
            }
        }
    }
```

bank-account/account.cmd/src/main/java/com/techbank/account/cmd/infrastructure/AccountEventStore.java

```
@@ -9,6 +9,7 @@ import com.techbank.cqrs.core.exceptions.ConcurrencyException;
 import com.techbank.cqrs.core.infrastructure.EventStore;
 import com.techbank.cqrs.core.producers.EventProducer;
 import org.springframework.beans.factory.annotation.Autowired;
+import org.springframework.beans.factory.annotation.Value;
 import org.springframework.stereotype.Service;

 import java.util.Date;
@@ -17,6 +18,9 @@ import java.util.stream.Collectors;

 @Service
 public class AccountEventStore implements EventStore {
+    @Value("${spring.kafka.topic}")
+    private String topic;
+
     @Autowired
     private EventProducer eventProducer;

@@ -43,7 +47,7 @@ public class AccountEventStore implements EventStore {
                     .build();
             var persistedEvent = eventStoreRepository.save(eventModel);
             if (!persistedEvent.getId().isEmpty()) {
-                eventProducer.produce(event.getClass().getSimpleName(), event);
+                eventProducer.produce(topic, event);
             }
         }
     }
```

bank-account/account.cmd/src/main/resources/application.yml

```
@@ -8,6 +8,7 @@ spring:
       port: 27017
       database: bankAccount
   kafka:
+    topic: BankAccountEvents
     producer:
       bootstrap-servers: localhost:29092
       key-serializer: org.apache.kafka.common.serialization.StringSerializer
```

bank-account/account.query/src/main/java/com/techbank/account/query/infrastructure/consumers/AccountEventConsumer.java

```
@@ -1,46 +1,40 @@
 package com.techbank.account.query.infrastructure.consumers;

-import com.techbank.account.common.events.AccountClosedEvent;
-import com.techbank.account.common.events.AccountOpenedEvent;
-import com.techbank.account.common.events.FundsDepositedEvent;
-import com.techbank.account.common.events.FundsWithdrawnEvent;
 import com.techbank.account.query.infrastructure.handlers.EventHandler;
+import com.techbank.cqrs.core.events.BaseEvent;
 import org.springframework.beans.factory.annotation.Autowired;
 import org.springframework.kafka.annotation.KafkaListener;
 import org.springframework.kafka.support.Acknowledgment;
 import org.springframework.messaging.handler.annotation.Payload;
 import org.springframework.stereotype.Service;

+import java.lang.reflect.InvocationTargetException;
+import java.lang.reflect.Method;
+import java.util.logging.Level;
+import java.util.logging.Logger;
+
 @Service
 public class AccountEventConsumer implements EventConsumer {
+    private final Logger logger = Logger.getLogger(AccountEventConsumer.class.getName());
+
     @Autowired
     private EventHandler eventHandler;

-    @KafkaListener(topics = "AccountOpenedEvent", groupId = "${spring.kafka.consumer.group-id}")
-    @Override
-    public void consume(@Payload AccountOpenedEvent event, Acknowledgment ack) {
-        this.eventHandler.on(event);
-        ack.acknowledge();
-    }
-
-    @KafkaListener(topics = "FundsDepositedEvent", groupId = "${spring.kafka.consumer.group-id}")
-    @Override
-    public void consume(@Payload FundsDepositedEvent event, Acknowledgment ack) {
-        this.eventHandler.on(event);
-        ack.acknowledge();
-    }
-
-    @KafkaListener(topics = "FundsWithdrawnEvent", groupId = "${spring.kafka.consumer.group-id}")
-    @Override
-    public void consume(@Payload FundsWithdrawnEvent event, Acknowledgment ack) {
-        this.eventHandler.on(event);
-        ack.acknowledge();
-    }
-
```

```
-    @KafkaListener(topics = "AccountClosedEvent", groupId = "${spring.kafka.consumer.group-id}")
+    // Single topic -> Kafka keeps the order of the events within a partition.
+    @KafkaListener(topics = "${spring.kafka.topic}", groupId = "${spring.kafka.consumer.group-id}")
     @Override
-    public void consume(@Payload AccountClosedEvent event, Acknowledgment ack) {
-        this.eventHandler.on(event);
+    public void consume(@Payload BaseEvent event, Acknowledgment ack) {
+        logger.info("Consumed " + event.getClass().getSimpleName() + " v" + event.getVersion() + " for account " + event.getI
d());
+        try {
+            // Dispatch to the EventHandler.on(...) overload matching the concrete event type
+            Method on = EventHandler.class.getMethod("on", event.getClass());
+            on.invoke(eventHandler, event);
+        } catch (NoSuchMethodException e) {
+            logger.log(Level.WARNING, "No event handler registered for " + event.getClass().getName());
+        } catch (IllegalAccessException | InvocationTargetException e) {
+            logger.log(Level.SEVERE, "Error while handling event " + event.getClass().getName(), e);
+            return; // no ack: the same event is redelivered, order is preserved
+        }
         ack.acknowledge();
     }
 }
```

bank-account/account.query/src/main/java/com/techbank/account/query/infrastructure/consumers/EventConsumer.java

```
@@ -1,15 +1,9 @@
 package com.techbank.account.query.infrastructure.consumers;

-import com.techbank.account.common.events.AccountClosedEvent;
-import com.techbank.account.common.events.AccountOpenedEvent;
-import com.techbank.account.common.events.FundsDepositedEvent;
-import com.techbank.account.common.events.FundsWithdrawnEvent;
+import com.techbank.cqrs.core.events.BaseEvent;
 import org.springframework.kafka.support.Acknowledgment;
 import org.springframework.messaging.handler.annotation.Payload;

 public interface EventConsumer {
-    void consume(@Payload AccountOpenedEvent event, Acknowledgment ack);
-    void consume(@Payload FundsDepositedEvent event, Acknowledgment ack);
-    void consume(@Payload FundsWithdrawnEvent event, Acknowledgment ack);
-    void consume(@Payload AccountClosedEvent event, Acknowledgment ack);
+    void consume(@Payload BaseEvent event, Acknowledgment ack);
 }
```

bank-account/account.query/src/main/java/com/techbank/account/query/infrastructure/handlers/AccountEventHandler.java

```
@@ -46,6 +46,7 @@ public class AccountEventHandler implements EventHandler {
         }
         var currentBalance = bankAccount.get().getBalance();
         var latestBalance = currentBalance - event.getAmount();
+        bankAccount.get().setBalance(latestBalance);
         accountRepository.save(bankAccount.get());
     }
```

bank-account/account.query/src/main/resources/application.yml

```
@@ -14,6 +14,7 @@ spring:
       username: root
       password: techbankRootPsw
   kafka:
+    topic: BankAccountEvents
     listener:
       ack-mode: MANUAL_IMMEDIATE
     consumer:
```